Assignment 3 - README

Question 1 - Wireshark Labs (IP & ICMP)

Setup Requirements

- Install **Wireshark** from: `https://www.wireshark.org/download.html`
- Download the pre-captured trace files zip:

  http://gaia.cs.umass.edu/wireshark-labs/wireshark-traces-8.1.zip

- Extract the zip file. The two trace files needed are:
  - `ip-ethereal-trace-1` - for the IP lab
  - The ICMP trace file - for the ICMP lab

Part A - IP Wireshark Lab

Trace file: `ip-ethereal-trace-1`
Lab reference: `https://t.ly/Syyf`

Procedure

Step 1 - Load the trace file

File -> Open -> ip-ethereal-trace-1

Step 2 - Apply ICMP filter
Enter the following in Wireshark's display filter bar and press **Enter**:

icmp

This filters out all non-ICMP traffic.
Step 3 - Select the first ICMP Echo Request (Frame 8)
Click on the first packet showing Protocol `ICMP` and Info `Echo (ping) request`.
In the packet details pane, expand:

Internet Protocol Version 4

All IP header fields will be displayed.
Step 4 - Extract basic IP header information (Q1-Q4)
From the expanded IP header, locate:

- **Source Address** → Answer for Q1 (your host IP)
- **Protocol** field → Answer for Q2
- **Header Length** and **Total Length** → Compute `Payload = Total - Header` → Answer for Q3
- **Flags** (0x00) and **Fragment Offset** (0) → Confirms no fragmentation → Answer for Q4

Step 5 - Compare fields across multiple packets (Q5-Q7)
Examine several consecutive Echo Request packets with the IP header expanded:

- Fields that **change** each packet: TTL, Identification, Header Checksum

- Fields that **remain constant**: Source IP, Destination IP, Version, Header Length, Protocol
- Note that the Identification field increments by 1 per packet → Answer for Q7

Step 6 - Examine ICMP TTL-exceeded reply packets (Q8-Q9)
Clear the filter and locate packets with Info `Time-to-live exceeded`.
Select one, expand its IP header, and note:

- **Identification** and **TTL** values → Answer for Q8
- Compare across multiple TTL-exceeded packets:
  - TTL stays constant (determined by the router's OS)
  - Identification changes (each reply is a new datagram from the router) → Answer for Q9

Step 7 - Analyze 2-fragment packets (Q10-Q13)
Find packet groups labeled:

```
[2 IPv4 Fragments (2008 bytes): #92(1480), #93(528)]
```

These are already present in the pre-captured trace - no manual generation needed.

- Select the **first fragment** (Fragment Offset = 0, MF flag = 1) → Answers for Q10, Q11
- Select the **second fragment** (Fragment Offset = 1480, MF flag = 0) → Answer for Q12
- Compare IP header fields between fragments:
  - Fields that differ: Fragment Offset, MF flag, Total Length, Header Checksum
  - Fields that match: Identification, Source IP, Destination IP → Answer for Q13

Step 8 - Analyze 3-fragment packets (Q14)
Locate packet groups labeled:

```
[3 IPv4 Fragments (3508 bytes): #216(1480), #217(1480), #218(548)]
```

These are also included in the trace file.

- Examine all three fragments and compare their IP header fields → Answer for Q14

Part B - ICMP Wireshark Lab

Trace file: ICMP trace file
Lab reference: `https://t.ly/R7Vk`

Procedure

Step 1 - Load the trace file

```
File -> Open -> <icmp-trace-file>
```

Step 2 - Apply ICMP filter
In the display filter bar, type:

```
icmp
```

and press **Enter**.
Step 3 - Identify host and destination IPs (Q1)
Click on the first `Echo (ping) request` packet.
Expand `Internet Protocol Version 4` and locate:

- **Source Address** = your host IP
- **Destination Address** = ping target IP → Answer for Q1

Step 4 - Note the lack of port numbers (Q2)
Expand `Internet Control Message Protocol` - no source/destination port fields exist.
ICMP operates at Layer 3 (Network layer), not Layer 4, so port numbers are not used → Answer for Q2

Step 5 - Examine ICMP Echo Request fields (Q3)
With the Echo Request selected, expand `Internet Control Message Protocol`:

- **Type = 8**, **Code = 0**
- Additional fields: Checksum, Identifier, Sequence Number, Data → Answer for Q3

Step 6 - Examine ICMP Echo Reply fields (Q4)
Click on the corresponding `Echo (ping) reply` packet.
Expand `Internet Control Message Protocol`:

- **Type = 0**, **Code = 0**, remaining fields identical → Answer for Q4

Step 7 - Isolate traceroute packets (Q5-Q7)
In the filter bar, enter:

```
icmp && ip.src == 192.168.1.101
```

Click the first traceroute Echo Request and identify:

- New **Destination IP** → Answer for Q5
- **Protocol** field in the IP header (still 1 = ICMP, not 17 = UDP) → Answer for Q6
- Compare **TTL** values - traceroute uses small TTL values (1, 2, 3...) vs. large TTL in normal ping → Answer for Q7

Step 8 - Examine Time Exceeded error packets (Q8)
Revert filter to `icmp` and select a packet with Info `Time-to-live exceeded`.
Expand `Internet Control Message Protocol` - note the embedded fields:

- **Original IP header** (Source + Destination of the probe packet)
- **First 8 bytes of the original ICMP payload** (Identifier + Sequence Number) → Answer for Q8

Step 9 - Compare final three packets with earlier ones (Q9)
Scroll to the last three packets in the ICMP list:

- Last three: `Echo (ping) reply` - Type 0, Code 0 - sent by the **destination host**
- Earlier packets: `Time-to-live exceeded` - Type 11, Code 0 - sent by **intermediate routers** → Answer for Q9

Step 10 - Identify the link with longest delay (Q10)
Examine the Time column for consecutive TTL-exceeded packets from different hops.
Find where the RTT shows a sharp increase:

- Hop 9: `att-gw.nyc.opentransit.net` - New York, USA (∼25 ms)
- Hop 10: `Pastourelle.opentransit.net` - Paris, France (∼98 ms)

This large increase is due to a transatlantic undersea cable link → Answer for Q10

Question 2 - Centralized and Distributed Routing Algorithms

Part 1: Link-State Routing (Dijkstra's Algorithm)

Execution:

```
python3 part1_q2.py
```

Expected output (summary):

```
Node 0 -> Dest 1: cost 1, next hop 1
Node 0 -> Dest 2: cost 2, next hop 1
Node 0 -> Dest 3: cost 4, next hop 1
...
```

Part 2 (Optional): Distance-Vector Routing (Bellman-Ford)

Execution:

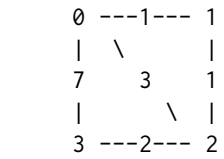
```
python3 part2_q2.py
```

Expected final routing tables (after convergence):

```
Node 0 -> Dest 1: next hop 1, cost 1
Node 0 -> Dest 2: next hop 1, cost 2
Node 0 -> Dest 3: next hop 1, cost 4
...
```

Both Part 1 and Part 2 yield identical final routing tables.

Network Topology (Question 2)



Link costs (bi-directional):
`c(0,1)=1, c(0,2)=3, c(0,3)=7, c(1,2)=1, c(2,3)=2`
Nodes 1 and 3 are not directly connected.

Question 3 - HOL Blocking

Execution:

```
python3 q3.py
```

Expected output (key results):

```
\pi(11) = \pi(12) = \pi(21) = \pi(22) = 0.25
Average Throughput (Analytical): 1.500000 packets/slot
Efficiency: 75.0% | HOL loss: 25.0%
Verification \pi P = \pi: True
```